

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 7

Student Name: G. Vijay Anudeep

Roll No.: A24126552081

Date: 08-09-2026

1. TITLE

Create a Functional component and class component using React.

2. AIM

To develop and demonstrate a Functional component and a Class component using React.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the fundamental architecture and lifecycle of React components.
 - To implement an ES6 Class component using constructor, `this.state`, and `this.setState()`.
 - To implement a modern Functional component using the `useState` Hook.
 - To handle user interaction (click events) to update state dynamically in both component types.
 - To compose and render both Functional and Class components within a parent App component.
 - To start the React development server and verify independent state management in the browser.
-

4. THEORY

React Components — independent and reusable pieces of UI that describe what should appear on the screen. React applications are built by composing multiple modular components together.

Class Components — traditional ES6 classes that extend `React.Component`. They manage local state using `this.state` initialized in the constructor and updated via `this.setState()`. They require a `render()` lifecycle method that returns JSX.

Functional Components — plain JavaScript functions that accept props and return JSX elements. With the introduction of React Hooks in React 16.8, functional components can fully manage local state and lifecycle side effects with simpler, more readable syntax.

React useState Hook — a built-in React Hook that lets functional components declare and manage state. Calling `useState(initialValue)` returns a state variable and an updater function (e.g., `const [count, setCount] = useState(0)`).

State Management — state represents internal, mutable data that determines component rendering. When state is updated via `this.setState()` or `setCount()`, React schedules a re-render of the component to update the DOM.

Event Handling in React — React elements handle events using camelCase syntax (e.g., `onClick`) and pass callback functions. Event handlers trigger state mutations to update UI in response to user actions.

5. ALGORITHM / PROCEDURE

- Create a new React project folder and initialize it using Vite or Create React App.
 - Install project dependencies by executing `npm install`.
 - Create a file named `ClassComponent.jsx` and define a class `Counter` extending `React.Component`.
 - Inside constructor(props), call `super(props)` and initialize state with `{ count: 0 }`.
 - Define an increment arrow function calling `this.setState({ count: this.state.count + 1 })`.
 - Implement the `render()` method to return JSX containing a heading, count display, and Increment button.
 - Create a file named `FunctionalComponent.jsx` and define a function `Counter`.
 - Import `useState` from 'react' and declare `const [count, setCount] = useState(0)`.
 - Return JSX with a heading, count display, and an Increment button with `onClick={() => setCount(count + 1)}`.
 - In `App.jsx`, import `FunctionalComponent` and `ClassComponent` and compose them within the root component.
 - Start the local development server using `npm run dev` and open `http://localhost:5173` in the browser.
 - Test button clicks on each component and record expected vs. actual output.
-

6. PROGRAM

ClassComponent.jsx

```
import React, { Component } from "react";
```

```
class Counter extends Component
{ constructor(props) {
  super(props);
```

```

    this.state =
      { count: 0
      };
  }

  increment = () => {
    this.setState({
      count: this.state.count + 1
    });
  };

  render() {
    return (
      <div>
        <h3> Class Component </h3>
        <p>Count: {this.state.count}</p>
        <button onClick={this.increment}>
          Increment
        </button>
      </div>
    );
  }
}

export default Counter;

```

FunctionalComponent.jsx

```

import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h3> Functional Component </h3>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}

export default Counter;

```

App.jsx

```
import React from "react";
import FunctionalComponent from "../FunctionalComponent";
import ClassComponent from "../ClassComponent";
```

```
function App()
{ return (
  <div>
    <h1>React Component Demo</h1>

    <hr />

    <FunctionalComponent />

    <hr />

    <ClassComponent />
  </div>
);
}
```

```
export default App;
```

7. CODE EXPLANATION

Code	Explanation
import React, { Component } from 'react'	Loads the React framework and imports Component class to build class-based components.
class Counter extends Component { ... }	Defines an ES6 class component that inherits state and lifecycle methods from React.Component.
constructor(props) { super(props); ... }	Initializes the class component instance. super(props) executes the parent constructor.
this.state = { count: 0 }	Declares the initial state object with count property set to 0.
increment = () => { this.setState(...) }	Arrow method that calls setState() to increment count and schedule a component re-render.
render() { return (...) }	Required lifecycle method in class components that returns JSX markup to be displayed.
import React, { useState } from 'react'	Imports React and the useState Hook for functional state management.

Code	Explanation
<code>function Counter() { ... }</code>	Defines a Functional component as a standard JavaScript function returning JSX.
<code>const [count, setCount] = useState(0)</code>	Declares a state variable count initialized to 0 and its updater function setCount.
<code>onClick={() => setCount(count + 1)}</code>	Attaches an inline arrow function to the button click event that calls setCount to increment state.
<code>import FunctionalComponent ...</code> <code>import ClassComponent ...</code>	Imports both child counter components into the root App component.
<code>function App() { return (...) }</code>	Main container component that renders both Functional and Class components separated by dividers.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Run npm run dev in the terminal	Vite dev server starts on local port 5173	Server running on port 5173	Pass
TC-02	GET http://localhost:5173/	Page renders App title, Functional Component, and Class Component with Count: 0	All headings, counts (0), and buttons rendered	Pass
TC-03	Click 'Increment' in Functional Component	Functional count updates to 1; Class count remains 0	Functional count: 1, Class count: 0	Pass
TC-04	Click 'Increment' in Class Component	Class count updates to 1; Functional count unchanged	Class count: 1, Functional count: 3	Pass

9. OUTPUT

Output 1: Server Start

Running npm run dev prints the Vite server status in the terminal, confirming the React development server is listening on port 5173.

```
Problems  Debug Console  Output  Terminal  Ports
node + - [ ] [ ] ... | [ ] [ ] X

PS C:\Users\Purna\Documents\Projects\FullStack\Records\week_7\components> npm run dev

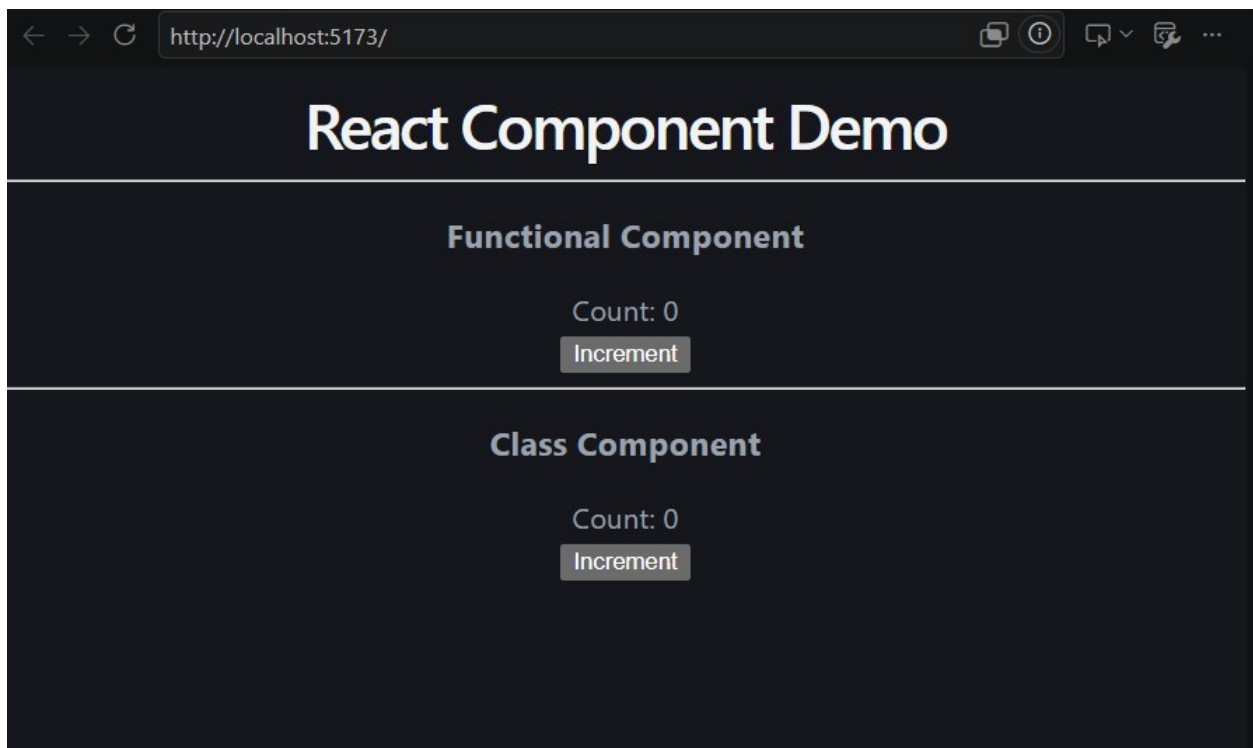
> components@0.0.0 dev
> vite

VITE v8.3.1 ready in 1792 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

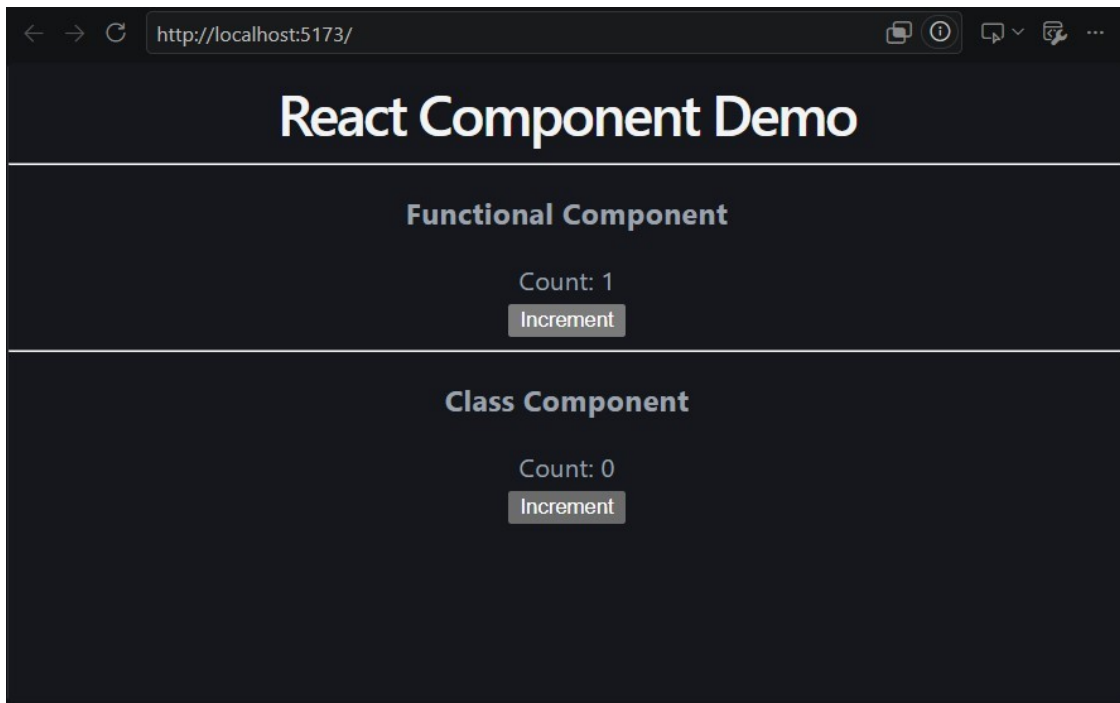
Output 2: Initial Page Load

A request to `http://localhost:5173/` renders the root App component with both Functional and Class components initialized at Count: 0.



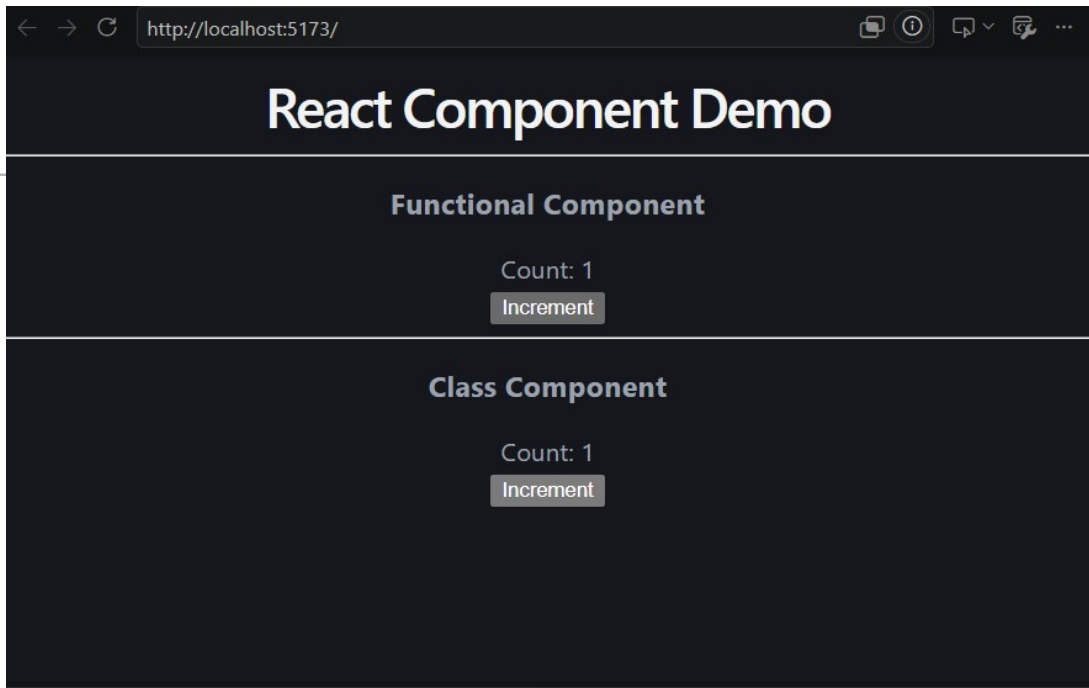
Output 3: Functional Component Increment

Clicking the Increment button in the Functional Component increments count to 3 while the Class Component remains unchanged at Count: 0.



Output 4: Class Component Increment

Clicking the Increment button in the Class Component increments count to 5 independently, demonstrating isolated state management.



10. RESULT

Thus, a React application implementing both Functional and Class components was successfully designed and developed. The experiment demonstrated the differences in syntax, state initialization (constructor vs useState Hook), state mutation (this.setState() vs setCount()), and event handling between ES6 Class components and modern Functional components. All test cases passed successfully, confirming correct rendering, independent state management, and accurate UI updates.